MACHINE LEARNING (23A31401T)

UNIT – IV : Linear Discriminants, SVM & MLP

Easy digital notes with diagrams for students

Topics in this unit

1. Introduction to Linear Discriminants (hyperplane, w.x + b)
2. Linear Discriminants for Classification
3. Perceptron Classifier
4. Perceptron Learning Algorithm (AND-gate example)
5. Support Vector Machines (maximum margin)
6. Linearly Non-Separable Case (soft margin)
7. Non-linear SVM
8. Kernel Trick (polynomial, RBF)
9. Logistic Regression
10. Linear Regression
11. Multi-Layer Perceptrons (MLPs)
12. Back propagation for training an MLP

> * **Study tip:**
> Every topic has a small diagram. Learn the RED words first, then draw the diagram once from memory - that is the fastest way to remember it for exams. Practise the three numeric examples (AND perceptron, regression line, one backprop step) twice.

Syllabus (as given)

Linear Discriminants for Machine Learning: **Introduction to Linear Discriminants, Linear Discriminants for Classification, Perceptron Classifier, Perceptron Learning Algorithm, Support Vector Machines, Linearly Non-Separable Case, Non-linear SVM, Kernel Trick, Logistic Regression, Linear Regression, Multi-Layer Perceptrons (MLPs), Back propagation for Training an MLP.**

Part I : Linear Discriminants & the Perceptron

4.1 Introduction to Linear Discriminants

A **linear discriminant** is a function that separates classes using a **straight boundary**. It is the simplest and fastest way to classify.

- Score: **$g(x) = w.x + b$** (w = weight vector, b = bias).
- In 2-D the boundary **$w.x + b = 0$** is a line; in 3-D a plane; in n-D a **hyperplane**.
- The weight vector **w is perpendicular (normal)** to the hyperplane.
- Sign of $g(x)$ tells the side: positive -> class +1, negative -> class -1.
- **b** shifts the line away from the origin; distance of the plane from origin = $|b| / ||w||$.

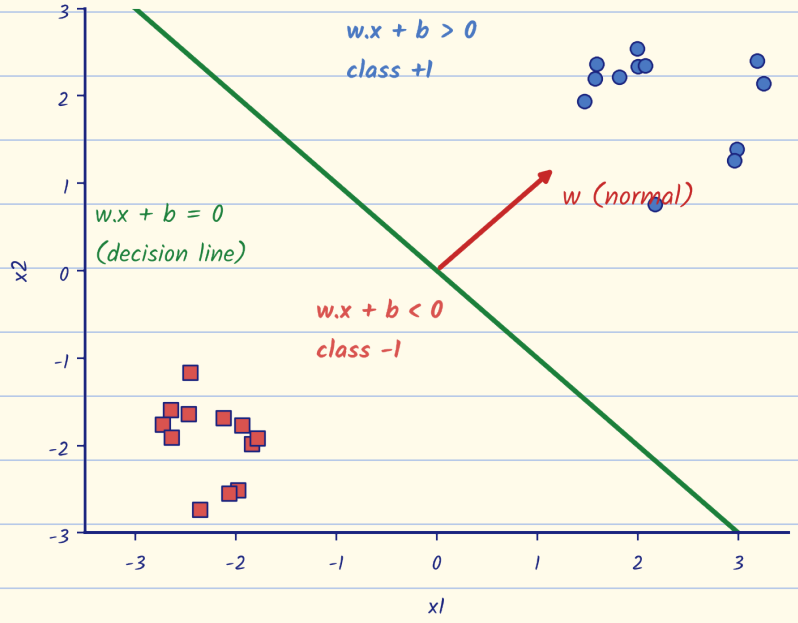


Fig 4.1 : Linear separator, its normal vector w and the two sides

> * **Remember:**
> Linear discriminant = **$w.x + b$**. Hyperplane = set of points where it equals 0. Distance of a point x from it = **$|w.x + b| / ||w||$**.

4.2 Linear Discriminants for Classification

- **Two classes:** predict +1 if $w.x + b > 0$, else -1. One hyperplane is enough.
- **K classes (one-vs-rest):** train K discriminants $g_k(x) = w_k.x + b_k$ and choose the class with the **largest score**.

- **One-vs-one:** train a classifier for every pair of classes (K(K-1)/2) and take a majority vote.
- Data that can be split by a hyperplane is called **linearly separable**.
- Ways to learn w and b: perceptron rule, SVM (max margin), logistic regression, least squares.

Method	Idea
Perceptron	Any separating line; updates only on mistakes
SVM	Best line = **widest margin**
Logistic regression	Gives a **probability** using sigmoid
Least squares	Minimise squared error of w.x + b

4.3 Perceptron Classifier

The **perceptron** (Rosenblatt, 1958) is the simplest artificial neuron: a weighted sum followed by a **step activation**.

- Net input: **net = w1x1 + w2x2 + ... + wnxn + b = w.x + b**.
- Output: **y = 1 if net > 0, else 0** (or +1 / -1).
- It is a **linear classifier** - it draws one hyperplane.
- **Limitation:** cannot solve **XOR** because XOR is not linearly separable. Fix: add hidden layers (MLP).

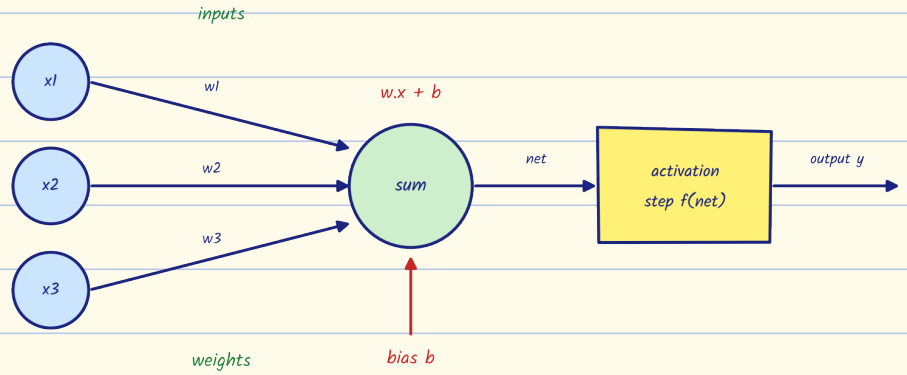


Fig 4.2 : Perceptron as a neuron (inputs, weights, sum, activation)

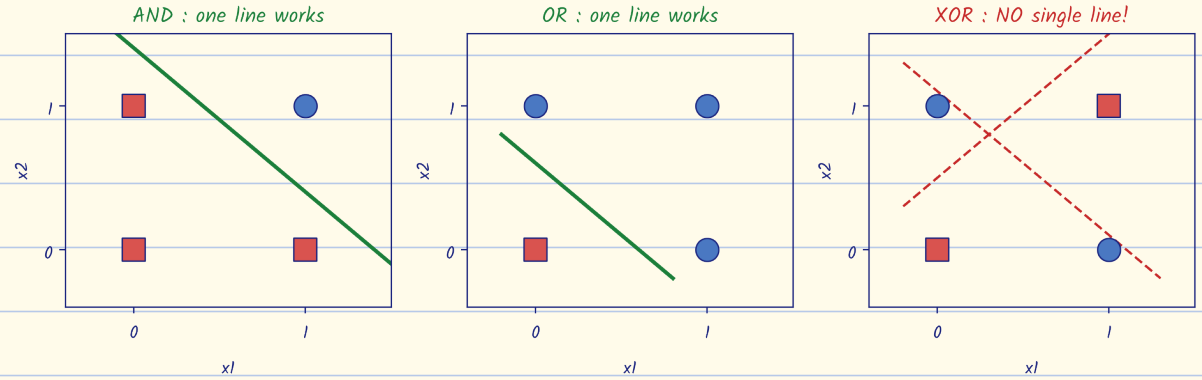


Fig 4.3 : AND and OR are separable by one line, XOR is not

4.4 Perceptron Learning Algorithm (PLA)

Start with small weights and **correct them only when a sample is misclassified**. Repeat over the data (each pass = **epoch**) till no errors.

1. Initialise w = 0, b = 0; choose learning rate **eta** (here 1).
2. For each sample (x, t): compute y = step(w.x + b).
3. Update: **w = w + eta (t - y) x** and **b = b + eta (t - y)**.
4. If y = t nothing changes. Stop after an epoch with **zero errors**.

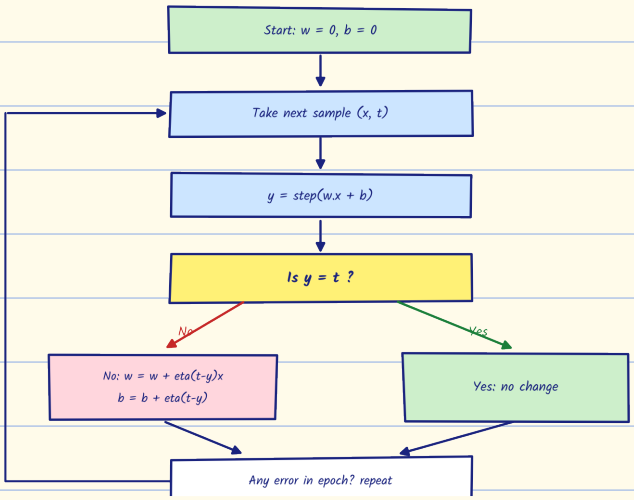


Fig 4.4 : Flowchart of the perceptron learning loop

* **Convergence theorem:**
If the data is **linearly separable**, PLA reaches a perfect separator in a **finite number of steps**. If not (like XOR), it never settles.

Worked example : learning the AND gate

Inputs (x1,x2), target t = x1 AND x2, eta = 1, start w1 = w2 = b = 0, rule: y = 1 if net > 0.

Epoch	(x1,x2)	t	net	y	t-y	new w1, w2, b
1	(0,0)	0	0	0	0	0, 0, 0
1	(0,1)	0	0	0	0	0, 0, 0
1	(1,0)	0	0	0	0	0, 0, 0
1	(1,1)	1	0	0	+1	1, 1, 1
2	(0,0)	0	1	1	-1	1, 1, 0
2	(0,1)	0	1	1	-1	1, 0, -1
2	(1,0)	0	0	0	0	1, 0, -1

Epoch	(x1,x2)	t	net	y	t-y	new w1, w2, b
2	(1,1)	1	0	0	+1	2, 1, 0
3	all 4 samples	-	-	-	3 errors	2, 1, -1
4	all 4 samples	-	-	-	2 errors	2, 2, -1
5	all 4 samples	-	-	-	1 error	2, 1, -2
6	all 4 samples	-	-	-	0 errors	2, 1, -2

Epoch 6 has **no errors**, so we stop: **w1 = 2, w2 = 1, b = -2**. Decision line: **2x1 + x2 - 2 = 0**. Check (1,1): net = 1 > 0 -> 1. Check (0,1): net = -1 -> 0.

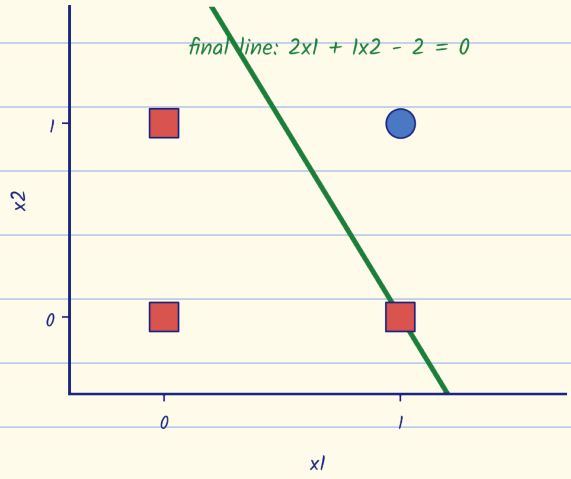


Fig 4.5 : Final AND decision line found by PLA

Part 2 : Support Vector Machines

4.5 Support Vector Machines (SVM)

Many lines can separate two classes. The **SVM chooses the best one - the line with the maximum margin** (largest gap to the nearest points). A wide gap means better generalisation.

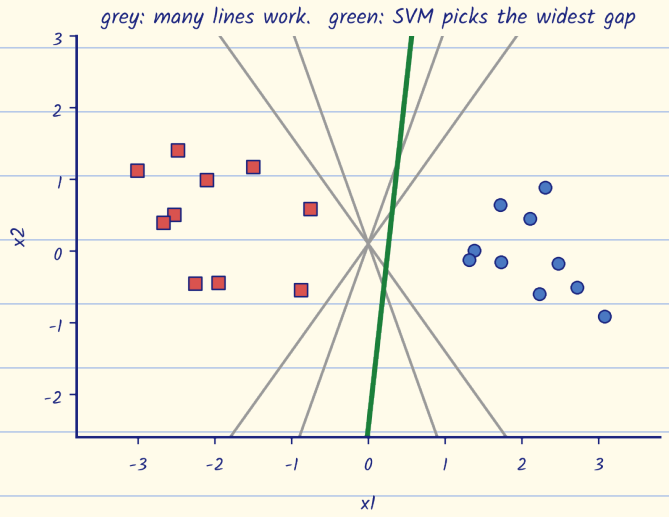


Fig 4.6 : Many separators exist, SVM picks the widest-gap one

- **Support vectors** = the training points **closest** to the boundary (on the margin lines). Only they decide the hyperplane; other points can be removed freely.
- Margin lines: **w.x + b = +1** and **w.x + b = -1**; boundary: w.x + b = 0.
- Constraint for every point: **t_i (w.x_i + b) >= 1** (t_i = +1 or -1).
- Width of the gap: **margin = 2 / ||w||**. So **maximise margin = minimise ||w||**.

> * **Optimisation problem (hard margin):**
> minimise **(1/2) ||w||^2** subject to **t_i (w.x_i + b) >= 1** for all i. It is a convex problem (quadratic programming) with a single global solution.

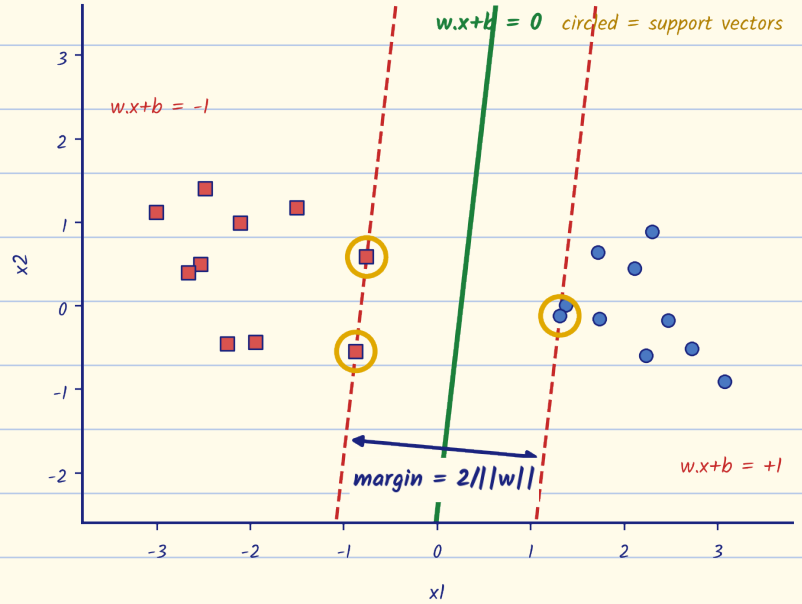


Fig 4.7 : Maximum margin, margin lines and support vectors (circled)

* **Example:**
If w = (3, 4) then ||w|| = sqrt(9 + 16) = 5, so **margin = 2/5 = 0.4**. A larger ||w|| means a thinner margin.

4.6 Linearly Non-Separable Case (Soft Margin)

Real data has noise and overlap, so a perfect line may not exist. The **soft-margin SVM** allows some points to break the rules, but charges a penalty.

- **Slack variable xi_i >= 0** = how much point i violates its margin (0 = fine, between 0 and 1 = inside margin, > 1 = misclassified).
- Constraint: **t_i (w.x_i + b) >= 1 - xi_i**.
- Objective: minimise **(1/2)||w||^2 + C * sum of xi_i**.
- **C** = trade-off knob between a wide margin and few errors.

C value	Effect
Large C	Few violations allowed, narrow margin, may **overfit**
Small C	More violations allowed, wide margin, may **underfit**

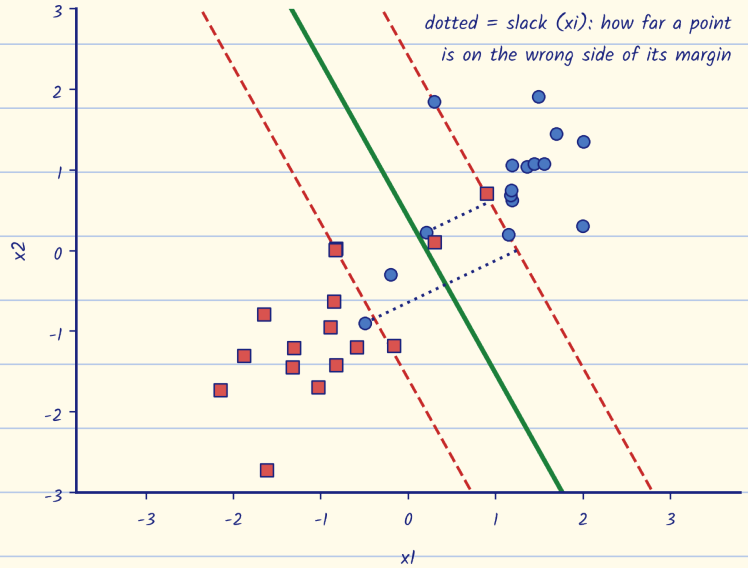


Fig 4.8 : Soft margin - slack (dotted) for points on the wrong side

4.7 Non-linear SVM

Some data (e.g. a ring of points around a blob) cannot be split by any line, even with slack. Idea: **map the data to a higher-dimensional space** where it becomes linearly separable, then use a linear SVM there.

- Feature map: **x -> phi(x)**. A linear boundary in phi-space is a **curved boundary** in the original space.
- Example: 2-D points (x1, x2) -> 3-D with extra feature **z = x1^2 + x2^2** (squared distance from centre).
- Inner circle has small z, outer ring has large z, so a **flat plane z = constant** separates them.

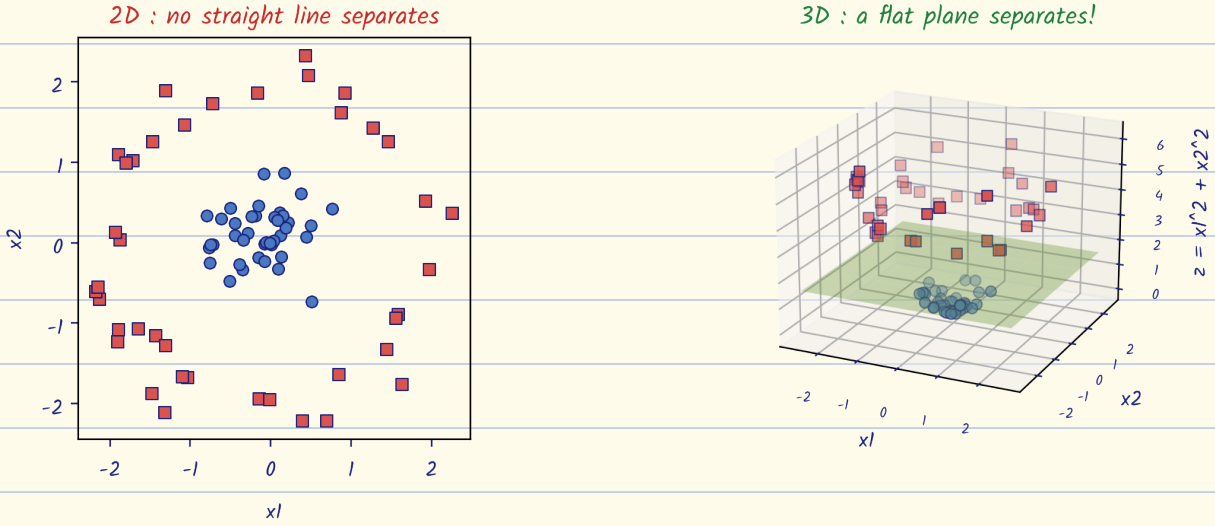


Fig 4.9 : 2-D data not separable, lifted to 3-D it is separable by a plane

4.8 Kernel Trick

Computing phi(x) explicitly can be huge (even infinite). The SVM only needs **dot products** phi(x).phi(z). A **kernel** gives that number directly from the original vectors.

> * **Kernel trick:**
> **K(x, z) = phi(x) . phi(z)**. Replace every dot product x.z in the SVM by K(x, z) - we get a non-linear classifier at the cost of a linear one. We never build phi(x).

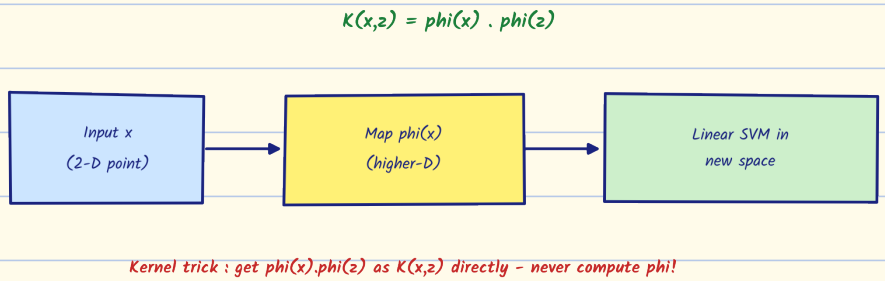


Fig 4.10 : Idea of the kernel trick

Kernel	Formula	Use
Linear	K = x.z	Linearly separable data

Kernel	Formula	Use
Polynomial	K = (x.z + c)^d	Curved, degree-d boundaries
RBF (Gaussian)	K = exp(-gamma ||x - z||^2)	Default choice; very flexible, local
Sigmoid	K = tanh(a x.z + c)	Neural-net like

- In RBF, **gamma** controls width: large gamma -> wiggly boundary (overfit), small gamma -> smooth.
- RBF kernel = 1 when x = z and falls towards 0 as points move apart (a similarity measure).

Worked example : kernel = dot product in a bigger space

Take x = (1, 2), z = (3, 1) and the degree-2 map **phi(v) = (v1^2, sqrt(2) v1 v2, v2^2)**.

Step	Calculation	Result
phi(x)	(1, sqrt2 * 1 * 2, 4)	(1.00, 2.83, 4.00)
phi(z)	(9, sqrt2 * 3 * 1, 1)	(9.00, 4.24, 1.00)
phi(x).phi(z)	9 + (2.83 x 4.24) + 4	**25.0**
Kernel (x.z)^2	x.z = 3 + 2 = 5, so 5^2	**25.0**
RBF, gamma = 0.5	||x-z||^2 = 4 + 1 = 5; exp(-0.5 x 5)	0.082

Both routes give **25** - but the kernel never left 2-D. That is the trick.

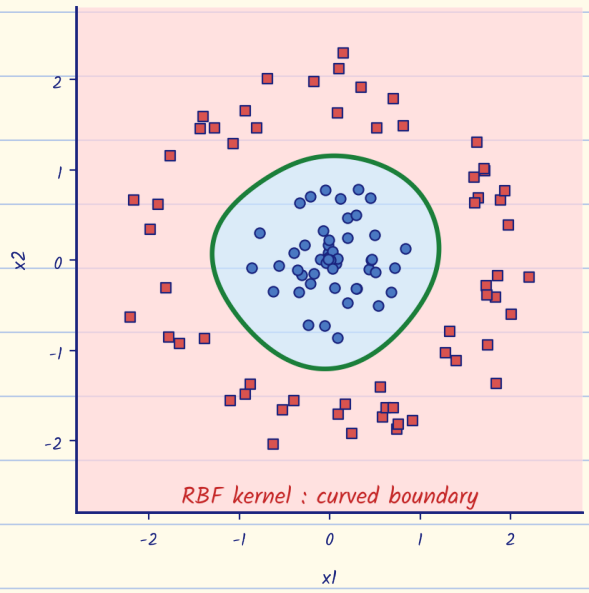


Fig 4.11 : RBF-kernel SVM draws a curved boundary around the inner class

Part 3 : Logistic & Linear Regression

4.9 Logistic Regression

Despite its name it is a **classification** method. It passes the linear score z = w.x + b through the **sigmoid** to get a probability between 0 and 1.

- Sigmoid: **p = 1 / (1 + e^(-z))**, where z = w.x + b.
- p = P(class 1 | x). Predict class 1 if **p >= 0.5** (i.e. z >= 0).
- Decision boundary is still **linear** (z = 0).
- Example: z = 0.8 gives p = **0.69** -> class 1.

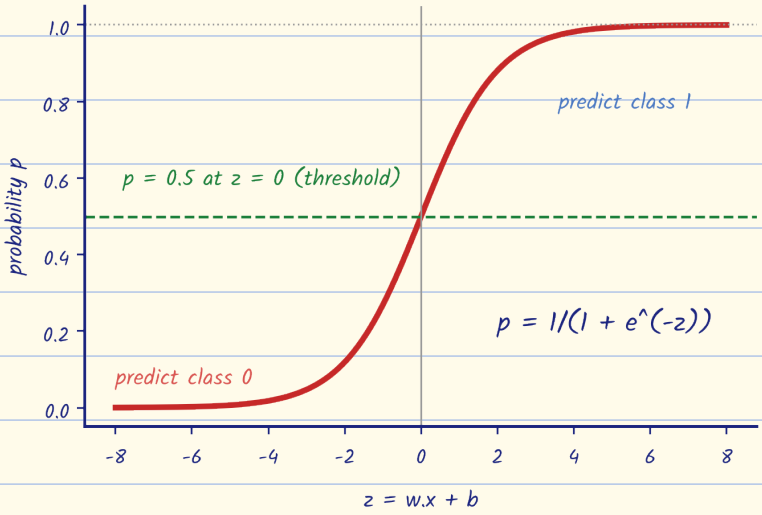


Fig 4.12 : Sigmoid squashes any score into a probability

Loss (log-loss / binary cross-entropy): for m samples with label y (0 or 1):

```
J(w,b) = -(1/m) * sum[ y*ln(p) + (1-y)*ln(1-p) ]
```

- If y = 1 and p is near 0, loss is huge; if p is near 1, loss is near 0 - it punishes confident wrong answers.
- Squared error would be non-convex here; log-loss is **convex**, so no bad local minima.
- **Gradient descent update:** w = w - alpha * (1/m) sum (p - y) x and b = b - alpha * (1/m) sum (p - y).
- Multi-class: **softmax** regression.

> * **Remember:**
> Logistic = linear score + sigmoid + log-loss + gradient descent. The gradient looks the same as in linear regression, only p replaces the prediction.

4.10 Linear Regression

Predicts a **continuous value** by fitting a straight line (hyperplane) to the data.

- Model: **y = w x + b** (or y = w.x + b for many inputs).
- **Residual** = actual y - predicted y.
- Least squares: choose w, b to minimise **SSE = sum (y - y_hat)^2** (or MSE = SSE / m).

Method	Formula / step
Simple formula	slope w = sum[(x - xbar)(y - ybar)] / sum[(x - xbar)^2]; b = ybar - w xbar
Normal equation	theta = (X^T X)^-1 X^T y (X has a column of 1s for b). Exact, but slow when features are many.
Gradient descent	w = w - alpha * (2/m) sum (y_hat - y) x. Repeat till loss stops falling. Works for huge data; needs a good alpha.

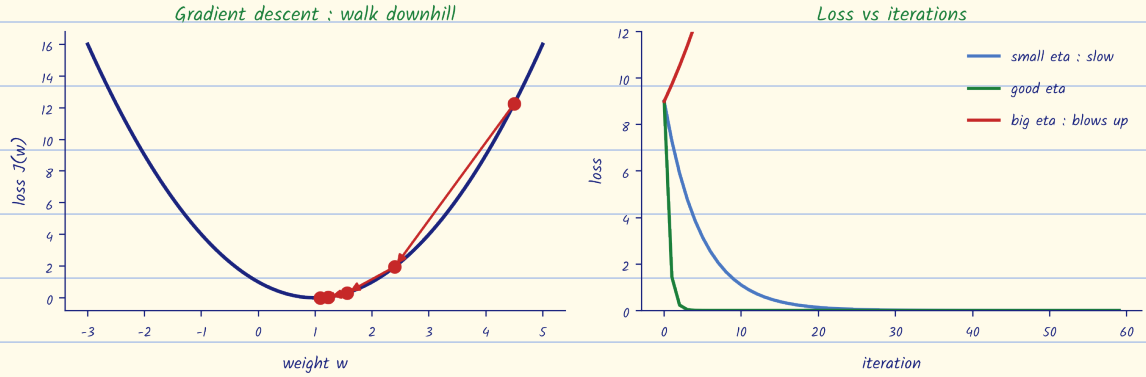


Fig 4.13 : Gradient descent walks downhill; learning rate decides the speed

Worked example : fit a line

Data (x, y): (1,2) (2,4) (3,5) (4,4) (5,5). xbar = 3, ybar = 4.

- sum (x-xbar)(y-ybar) = 4 + 0 + 0 + 0 + 4... computed exactly = **6**; sum (x-xbar)^2 = **10**.
- Slope w = 6/10 = **0.6**; intercept b = 4 - 0.6 x 3 = **2.2**. Line: **y = 0.6 x + 2.2**.
- Normal equation check: X^T X = [[5, 15],[15, 55]], X^T y = [20, 66]; theta = (2.2, 0.6). Same answer.
- Predictions: 2.8, 3.4, 4.0, 4.6, 5.2; residuals: -0.8, +0.6, +1.0, -0.6, -0.2; **SSE = 2.4**.
- Predict for x = 6: y = 0.6 x 6 + 2.2 = **5.8**.

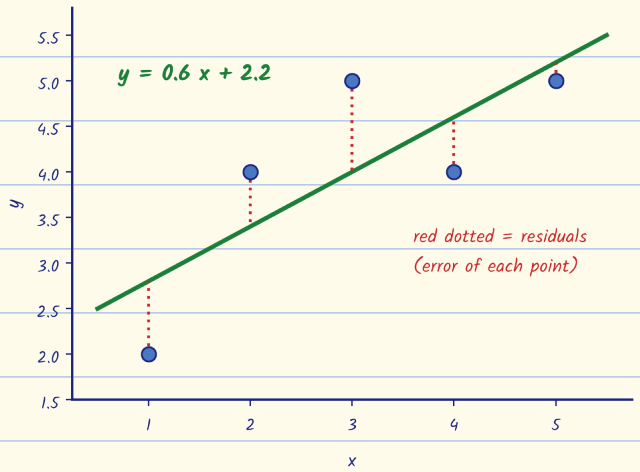


Fig 4.14 : Least-squares line and residuals

Part 4 : Multi-Layer Perceptrons & Backpropagation

4.11 Multi-Layer Perceptrons (MLPs)

A single perceptron cannot learn XOR. Stack neurons in layers and the network can learn **non-linear** functions. This is the **MLP**, a feed-forward neural network.

- **Input layer:** just holds the features (no computation).
- **Hidden layer(s):** neurons compute **h = f(W.x + b)**. Hidden layers learn new features.
- **Output layer:** gives the answer (sigmoid for binary, softmax for many classes, linear for regression).
- **Fully connected** and **feed-forward**: signals move only forward; every neuron links to all neurons of the next layer.
- Without a non-linear activation, many layers collapse into **one linear layer** - so activations are essential.
- **Universal approximation:** one hidden layer with enough neurons can approximate any continuous function.

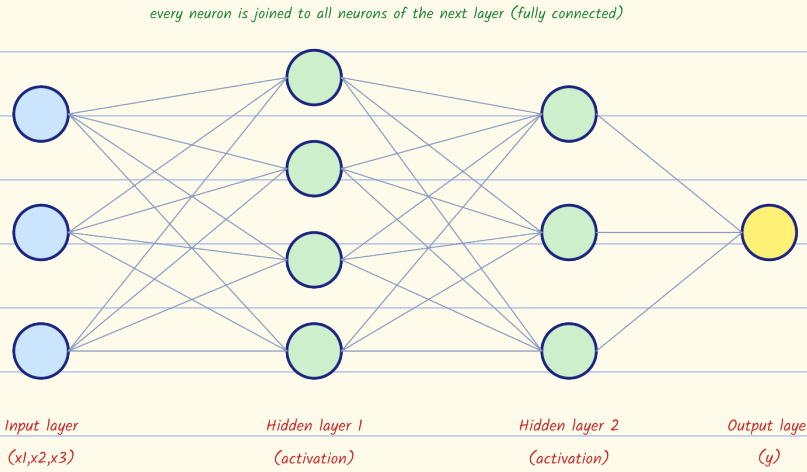


Fig 4.15 : A multi-layer perceptron (input, two hidden, output layers)

Activation	Formula	Output range / use
Step	1 if z > 0 else 0	0 or 1; perceptron, not differentiable
Sigmoid	1 / (1 + e^-z)	0 to 1; output of binary classifier
Tanh	(e^z - e^-z) / (e^z + e^-z)	-1 to 1; zero-centred
ReLU	max(0, z)	0 to inf; fast, most used in hidden layers

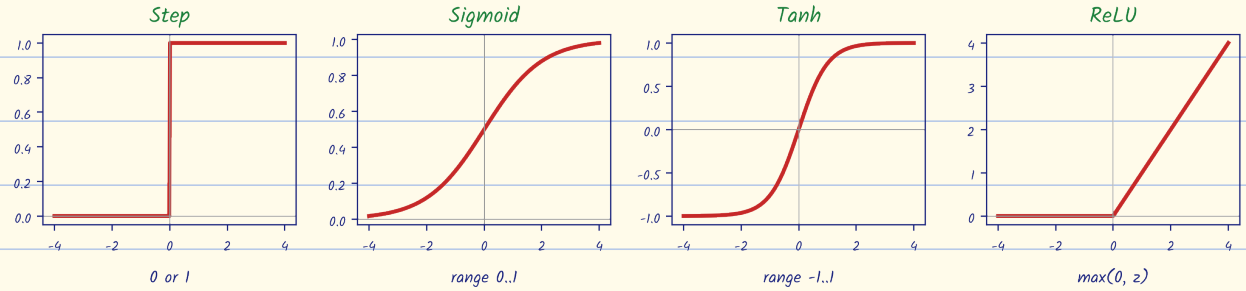


Fig 4.16 : Common activation functions

4.12 Back propagation for Training an MLP

Backpropagation is the algorithm that computes how much each weight contributed to the error, using the **chain rule**, and then updates the weights by **gradient descent**.

1. **Forward pass:** feed x through the layers to get output o.
2. **Error:** E = 1/2 (t - o)^2 (t = target).
3. **Backward pass:** use the chain rule to get dE/dw for every weight, from output layer back to the input.
4. **Update:** w = w - eta * dE/dw. Repeat for all samples / epochs.

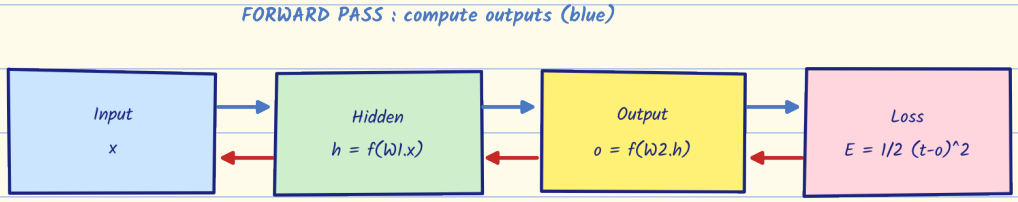


Fig 4.17 : Forward pass (blue) and backward pass (red)

For sigmoid units the chain rule gives (sigmoid derivative = o(1 - o)):

- Output delta: **d_o = (o - t) * o * (1 - o)**; gradient of weight h_j -> o is d_o * h_j.
- Hidden delta: **d_j = (w_jo * d_o) * h_j * (1 - h_j)**; gradient of weight x_i -> h_j is d_j * x_i.
- Bias gradient = the delta of that neuron.

Worked example : one step on a 2-2-1 network

Sigmoid everywhere, eta = 0.5, input x = (0.5, 1.0), target t = 1.

- Weights: x1->h1 = 0.1, x2->h1 = 0.3, x1->h2 = 0.2, x2->h2 = 0.4; biases of h1, h2 = 0.1; h1->o = 0.5, h2->o = 0.6; bias of o = 0.1.

Step	Calculation	Value
Forward: hidden	z_h1 = 0.1(0.5)+0.3(1.0)+0.1 = 0.45; z_h2 = 0.2(0.5)+0.4(1.0)+0.1 = 0.60	h1 = 0.6106, h2 = 0.6457
Forward: output	z_o = 0.5(0.6106)+0.6(0.6457)+0.1 = 0.7927	o = 0.6884
Error	E = 1/2 (1 - 0.6884)^2	E = 0.0485
Output delta	(o - t) o (1 - o) = (-0.3116)(0.6884)(0.3116)	d_o = -0.0668
Hidden deltas	d_1 = 0.5 d_o h1(1-h1); d_2 = 0.6 d_o h2(1-h2)	d_1 = -0.0079, d_2 = -0.0092
Update h->o	0.5 - 0.5(d_o h1); 0.6 - 0.5(d_o h2)	0.5204, 0.6216
Update x->h	w(x1->h1) = 0.1 - 0.5(d_1)(0.5); w(x2->h1) = 0.3 - 0.5(d_1)(1.0)	0.1020, 0.3040
New forward pass	same input, new weights	o = 0.7016, E = 0.0445

Output moved from **0.6884 to 0.7016** (towards t = 1) and error fell from **0.0485 to 0.0445** after a single update. Repeating this for many epochs trains the network.

* **Remember:**
Forward = compute. Backward = blame. Update = fix. Use a **small learning rate**; too big overshoots, too small is slow. Sigmoid/tanh can cause **vanishing gradients** in deep nets - ReLU helps.

Part 5 : Quick Revision & Likely Exam Questions

Quick Revision

Topic	One-line summary
Linear discriminant	g(x) = w.x + b; hyperplane g = 0; w is normal to it
Perceptron	Weighted sum + step; w = w + eta(t-y)x; fails on XOR
SVM	Maximum margin = 2/||w||; support vectors fix the boundary
Soft margin	Slack xi, penalty C: large C = narrow margin, small C = wide margin
Kernel trick	K(x,z) = phi(x).phi(z); poly (x.z + c)^d, RBF exp(-gamma||x-z||^2)
Logistic reg.	Sigmoid probability, log-loss, gradient descent, linear boundary
Linear reg.	Least squares: normal equation or gradient descent
MLP	Layers + non-linear activations; solves XOR
Backprop	Forward pass, error, chain-rule deltas backwards, w = w - eta dE/dw

Likely exam questions

1. What is a linear discriminant? Explain hyperplane and the role of w and b with a diagram.
2. Explain the perceptron and the perceptron learning algorithm. Train it for the AND gate (show epochs).
3. Why can a single perceptron not solve XOR? How does an MLP solve it?
4. Explain SVM. Derive / state margin = 2/||w|| and define support vectors.
5. What is the soft-margin SVM? Explain slack variables and the parameter C.
6. What is the kernel trick? Describe polynomial and RBF kernels with an example.
7. Explain logistic regression: sigmoid, log-loss and gradient descent. How is it different from linear regression?
8. Explain the backpropagation algorithm with a neat diagram and a numeric example for a 2-2-1 network.